

Sharding

Imagine Instagram has 500 million users. Can a single MongoDB server handle all the users, photos, comments, and messages forever?

No, Because a single server has limits:

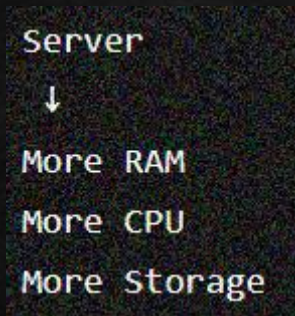
- ☐ Limited RAM
- ☐ Limited CPU
- ☐ Limited Storage
- ☐ Limited network capacity

This is called the **scaling problem**.

Vertical Scaling vs Horizontal Scaling

Vertical Scaling

Increase the power of one machine.

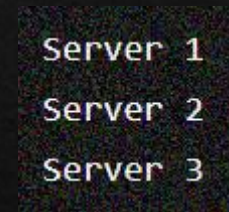


Example: 16 GB RAM → 64 GB RAM

Problem: Eventually, even the biggest server has limits and becomes very expensive.

Horizontal Scaling

Add more servers.



Instead of making one machine bigger, we use multiple machines.

This idea leads to **Sharding**

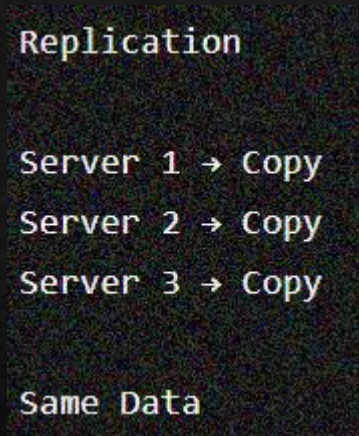
What is Sharding?

Sharding is the process of distributing data across multiple servers so that no single server stores all the data.

In simple words Replication copies data and Sharding divides data.

Replication vs Sharding

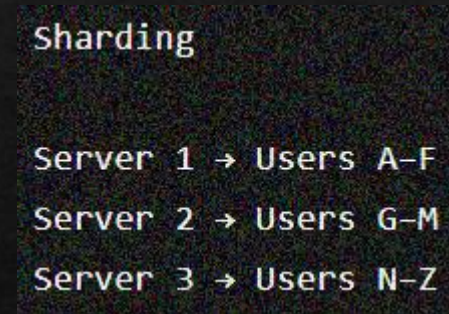
Replication



Purpose:

- ❑ High Availability
- ❑ Fault Tolerance

Sharding



Purpose:

- ❑ Handle huge amounts of data
- ❑ Scale horizontally

Sharded Cluster Architecture

MongoDB Sharding has three components:

Component 1: Shards

Shards store the actual data.

Component 2: Config Servers

Config Servers store metadata about the cluster.
(Metadata means : Information about where data lives)

Example:

```
User 123 → Shard 2
User 987 → Shard 1
```

Config servers know this. They do NOT store actual application data.

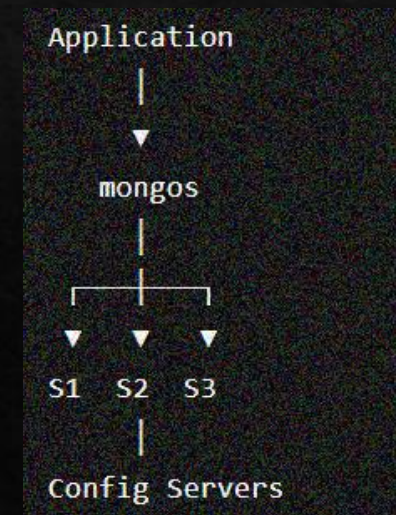
Component 3: mongos

mongos acts as a query router.

Applications connect to mongos.

They do NOT connect directly to shards.

Architecture Diagram:



How Query Routing Works

Step 1: Application sends query to mongos.

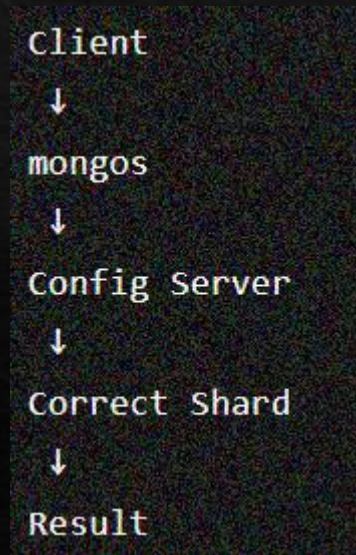
Step 2: mongos asks Config Servers : Which shard contains userId 500?

Step 3: Config Servers respond : Shard 2

Step 4: mongos sends query to Shard 2.

Step 5: Shard 2 returns result.

Query Flow:



What is a Shard Key?

A shard key is the field MongoDB uses to decide how to distribute data among shards.

```
{  
  userId: 1001,  
  name: "Manas"  
}
```

If `userId` is the shard key, MongoDB uses it to decide where this document goes.

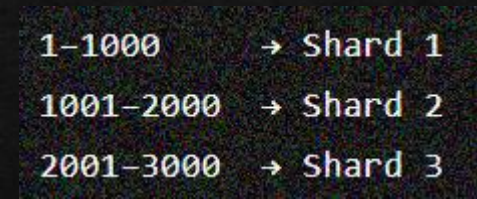
A good shard key has high cardinality. (many unique values)
for example: `userId`, `email`, `orderId`, etc.

Range-Based Sharding

Example:

Shard Key:userId

Distribution:

A screenshot of a terminal window with a dark background and light-colored text. It displays three lines of text showing the mapping of user ID ranges to shards.

1-1000	→ Shard 1
1001-2000	→ Shard 2
2001-3000	→ Shard 3

Advantage: Efficient range queries.

Problem: Can create hotspots.

Hashed Sharding

MongoDB hashes the shard key. (for ex: userId - 1 → has value 10283)

Distribution becomes random.

Advantage: Better distribution.

Problem: Less efficient for range queries.

Real-World Example

YouTube has Millions of videos.

Instead of storing everything on one database server:

```
Shard 1 → Some videos  
Shard 2 → Some videos  
Shard 3 → Some videos  
...
```

This allows YouTube to keep growing.

Understand Code:

```
sh.shardCollection(  
  "school.users",  
  { userId: "hashed" }  
)
```

sh stands for Sharding Helper.

It provides helper methods to manage a sharded cluster.

When you open mongosh, MongoDB automatically exposes some helper objects such as:

```
db    // Current database  
rs    // Replica set helper  
sh    // Sharding helper
```

Replication + Sharding Together

In production:

```
Shard 1 → Replica Set  
Shard 2 → Replica Set  
Shard 3 → Replica Set
```

So each shard is usually replicated.

This provides:

- ❑ Sharding → Scalability
- ❑ Replication → High Availability

Thank You